

Plexii — Feature & Module Inventory

A database seed, derived from the codebase

Commit	e5cd228f
Generated	2026-09-21
Rows	428
Source files	1,220 · 308,965 lines
Test files	935

The CSV is the deliverable. [docs/plexii-feature-inventory.csv](#) holds all 428 rows with the columns below, ready to load. This document explains what is in it and where each value came from — a seed nobody can audit gets distrusted the first time a row looks wrong.

Everything factual is read out of source by [scripts/extract-feature-inventory.mjs](#): the widget catalogue, the View union, the segment registry, the IPC registrations, the store and engine layers. Re-run it and the inventory matches the code again.

The written parts are marked. Positioning is judgement and cannot be derived, so it is confined to the feature areas later in this document.

There are no importance weights in here, deliberately. Assigning 1-10 by feel would put invented numbers into a database meant to inform real decisions, and they would be indistinguishable from measured ones a month later.

What you get instead are three measured signals you can weight yourself: **dependents** (how many files import it — the blast radius of a change), **surface** (how many operations a capability exposes), and **tests** (how many test files name it). The [weight](#) column is empty and yours.

The schema

Columns in `plexii-feature-inventory.csv`. "Derived" is read from code; "YOU" is left empty.

COLUMN	MEANING	SOURCE	HOW TO USE IT
<code>id</code>	Stable key, e.g. <code>widget.table</code>	Derived	Primary key. Stable across releases unless the thing is renamed in code.
<code>domain</code>	App / Screen / Widget / Capability / State / Engine	Derived	What kind of thing this row is.
<code>name</code>	Human name	Derived	The label the product itself uses where one exists.
<code>module</code>	Which part of the system it belongs to	Derived	Use for grouping and ownership.
<code>description</code>	What it is	Derived	From the catalogue hint or the module's own header comment — the code's words, not mine.
<code>status</code>	active / legacy	Derived	Legacy rows are still rendered for existing data but hidden from pickers.
<code>surface</code>	Operation count (capabilities)	Derived	How many IPC operations the namespace exposes. A rough proxy for how much of the product depends on it.
<code>callers</code>	Source files calling any of its functions	Derived	Engines only. ZERO means implemented but called by nothing that ships — see "Unwired" below.
<code>dependents</code>	Source files statically importing it	Derived	The change-impact column. Counts static imports only, so it can read low for a module reached another way — check callers alongside it.
<code>loc</code>	Lines of code	Derived	Size, not quality. Useful for spotting where complexity sits.
<code>tests</code>	Test files that mention it by name	Derived	A weak signal, and it UNDERCOUNTS: tests that refer to a thing by a different name are missed. It called calendar sync untested when it had five test files. Never act on it alone.
<code>weight</code>	Importance 1-10	YOU	Deliberately empty. See the note below.
<code>owner</code>	Who owns it	YOU	Empty.
<code>marketing_description</code>	Positioning copy	YOU	Empty per row; the areas worth selling are written up in this document.
<code>notes</code>	Anything else	YOU	Empty.

What is in it

DOMAIN	WHAT IT COVERS	ROWS	DERIVED FROM
App	Apps inside the four segments	37	Read from the segment registry (<code>segmentApps.ts</code>).
Screen	Every destination the app can navigate to	45	Read from the View union (<code>stores/view.ts</code>).
Widget	Building blocks you place on a desk	55	Read from the widget catalogue, including its written hint.
Capability	The IPC surface, grouped by namespace	100	Counted from <code>ipcMain.handle</code> registrations in <code>main</code> .
State	Renderer stores	71	Every store module, with how many files import it.
Engine	Substantial main-process modules (AI, database, mail)	120	Files over 80 lines, with dependants and size.

Cross-functionality

Links are not a separate table because they are already implicit in two columns and better recomputed than stored: `dependents` is the import graph, and a capability's namespace prefix maps to the stores and engines that call it. If you want an explicit edge list, the extractor already walks every file and can emit one — that is a small change to the script, not a new exercise.

The authored layer: positioning and change-impact notes for the areas worth selling and worth being careful with. Written, not derived.

The desk

Canvas · 55 widget kinds · layouts · wires

A canvas per piece of work, holding whatever that work needs — tasks, a table, a document, a live web page, an AI agent — side by side instead of scattered across apps.

Depends on: nodes, widgets, tables, documents
Impact of change: The most coupled surface in the product. A change to widget storage or layout reaches every desk and every widget kind; treat the widget contract as public API.

PlexiOffice

13 apps · 6 document types · Mail · Chat · Meet · Sign · Browser

Documents, spreadsheets, slides, diagrams, design and drawing in one place, plus the communication around them — and a web browser that does not need a desk.

Depends on: documents, mail, office shell, view store
Impact of change: Document types share one editor stack; a change to the editor reaches the standalone route, the desk widget and the Office shell at once.

Browsing agent

browserAgent · browserActions · browserOverlays · browserProgress · consent

Plexii browses for you — researching, comparing and extracting — visibly, one step at a time, stoppable, and with the results landing on a desk as a table you can use.

Depends on: Anthropic API, webview, consent store, tables
Impact of change: Per-site consent and the refusal rules are safety boundaries. Changing the action vocabulary changes what the model can do to a live page; the sanitiser is the gate.

AI assistant

anthropic · chat · action proposals · action executor

One assistant across the whole workspace that proposes changes you approve, rather than a chat box that writes text you then copy somewhere.

Depends on: Anthropic API, every store it can act on
Impact of change: The proposal vocabulary is the widest contract in the app: adding a verb adds something the AI can do everywhere, and every applier must handle it.

Mail

imap · mailPaging · mailTags · triage

Your inbox, filed against the work it belongs to — folders that relate to a desk or task, so a busy mailbox stops being one undifferentiated list.

Depends on: IMAP, nodes, AI triage
Impact of change: Paging is by UID rather than position; anything that assumes index ordering will break on a mailbox that changes under it.

Tables and data

tables · schema · rows · views (Table/Cards/Kanban/Calendar/Gantt)

A database that behaves like a spreadsheet, on a desk, fed by hand, by automations, or by what Plexii finds while browsing.

Depends on: tables store, widgets
Impact of change: Column types coerce values on write. Changing a type affects existing rows, so migrations matter more here than anywhere else.

Custom widgets

customWidget · sandbox · wizard · action vocabulary · helpers

Describe the tool you need and Plexii builds it, running sandboxed on your desk with access only to what you wire into it.

Depends on: Anthropic API, sandbox scheme, widget inputs
Impact of change: ADR-0009 is the boundary: unlimited in what a widget can BE, bounded in what it can REACH. Loosening the sandbox is a security change, not a feature change.

The tray and navigation

view store · openTray · segmentApps

Everything you have open in one strip — desks, documents, chats, apps — so moving between contexts is a click rather than a retrace.

Depends on: view store
Impact of change: The view union is the app's routing contract. A surface that holds "what am I looking at" in local state disappears from the tray, from Back, and from session restore.

Automation

flows · agents · wires · signals · streamdeck

Standing workers on your desks: rules that run, agents that watch, and physical buttons that fire them.

Depends on: flows, agents, widget links
Impact of change: Saved flows persist verb names. Renaming a verb silently changes what every stored flow does — SPEC-044 exists because of this.

Knowledge and search

knowledge · brain map · decisions · search

What the workspace knows, linked: entries, a graph of how they connect, the decisions taken and what a change puts at risk.

Depends on: knowledge store, search index
Impact of change: Decisions reference work items; deleting the referenced item must not orphan the decision.

People and organisation

contacts · directory · org admin · people map · presence

Who is in the workspace, how they relate, and who is around right now.

Depends on: contacts, org, presence

Impact of change: Org structure feeds permissions; a change to roles reaches sharing and the vault.

Sharing and collaboration

shares · liveDesk · crdt · collaborations

Hand someone a desk — including someone who is not a user yet — and work on it together, live.

Depends on: crdt, shares, auth

Impact of change: CRDT merge rules decide who wins a conflict. Changing them changes history, not just behaviour.

Unwired spec contracts 3

Engines whose exported functions no other source file calls. Measured by [callers](#), not inferred.

These are implemented and tested, and the commits that added them report the matching spec requirements as complete. But nothing in the running app calls them, so **the rules they encode do not govern the product that ships**. They are not dead code: each is a deliberate contract. What is missing is the wiring.

MODULE	WHAT IT IS MEANT TO ENFORCE	LINES	TEST MENTIONS
agents engine.agents	Agent governance (spec §78, §79, REQ-AGT). Agents act on behalf of a human and never exceed them: effective permissions are a subset of the principal's (AGT-001), every agent has exactly one accountable human (AGT-005), delegation never escalates (AGT-014).	166	14
evals engine.evals	AI evaluation framework (spec §70, §72, REQ-AI/AGT/ENG). Provider substitution is verifiable by an evaluation suite run against every supported model, and a provider is not "supported" without a passing run (AI-004).	107	1
resumeSummary engine.resumeSummary	The AI stage over a Resume (spec §52, §70) — deterministic first, AI second. The structured Resume is already complete; this stage adds prose over it, and it is safe by construction: - cached by the structured-input digest, so identical input never pays twice...	95	4

Agent governance: investigated, and the running agents ARE governed — by a different mechanism.

Every action a model proposes goes to a human for review; none is applied on its own. The one autonomous write a desk agent makes is a single [update-widget](#) to the one destination widget the user configured *and* explicitly set to auto-apply. Its kind and target are fixed in code — [buildDelivery](#) returns a type-narrowed update with the target id hard-coded — so the model controls only the text. AGT-001's intent holds structurally: an agent cannot do anything its human did not either approve or pre-authorise for one named widget.

So [ai/agents.ts](#) is a contract for a runtime that does not exist yet. Most of its fifteen requirements govern features the live system lacks: delegation between agents (AGT-014), schema-validated inter-agent messages (AGT-010/011/012), declared tool sets (AGT-020). The dispatcher's own header lists inter-agent dispatch as unbuilt Phase 3 work. The tracker marks these complete against capabilities that have not been built.

One path to decide on deliberately: autonomous external transmission. An agent on an interval trigger, auto-applying into its destination, with that destination wired into a "Send to a URL" widget, POSTs model-written content to an external address with no human in the loop. Verified, not inferred: the outbound-webhook spec passes 9/9, including a wired change POSTing its content.

Every link in that chain is something the user built, and auto-apply is an explicit opt-in, so it is consented. But AGT-021 says external transmission is *gated*, and here the gate is configuration rather than confirmation. That is a product decision, not a defect.

Apps inside the four segments 37

Read from the segment registry (segmentApps.ts).

NAME	MODULE	WHAT IT IS	TEST MENTIONS
Home app.plexidesk.home	plexidesk	The segment landing page	0
My Desk app.plexidesk.desk	plexidesk	An app inside the segment	3
Workspaces app.plexidesk.workspaces	plexidesk	An app inside the segment	0
Plans app.plexidesk.plans	plexidesk	An app inside the segment	1
Agentic Ops app.plexidesk.ops	plexidesk	An app inside the segment	0
Desks app.plexidesk.tasks	plexidesk	An app inside the segment	0
Calendar app.plexidesk.calendar	plexidesk	An app inside the segment	0
Files app.plexidesk.files	plexidesk	An app inside the segment	1
Recent app.plexidesk.recent	plexidesk	An app inside the segment	0
People Home app.plexipeople.home	plexipeople	The segment landing page	0
Directory app.plexipeople.directory	plexipeople	An app inside the segment	0
Organisation app.plexipeople.workspaces	plexipeople	An app inside the segment	0
Organisation Map app.plexipeople.map	plexipeople	An app inside the segment	0
Brain Home app.plexibrain.home	plexibrain	The segment landing page	0
Ask Brain app.plexibrain.ask	plexibrain	An app inside the segment	0
Search app.plexibrain.search	plexibrain	An app inside the segment	0
Brain Map app.plexibrain.map	plexibrain	An app inside the segment	0
Decisions app.plexibrain.decisions	plexibrain	An app inside the segment	0
Assemble a desk app.plexibrain.assemble	plexibrain	An app inside the segment	0
Flows app.plexibrain.flows	plexibrain	An app inside the segment	0
Agents app.plexibrain.agents	plexibrain	An app inside the segment	0
Connect app.plexibrain.connect	plexibrain	An app inside the segment	0
APIs app.plexibrain.api	plexibrain	An app inside the segment	0
Insights app.plexibrain.insights	plexibrain	An app inside the segment	0
Office Home app.office.home	office	The segment landing page	1
Mail app.office.mail	office	An app inside the segment	1
Inbox app.office.inbox	office	An app inside the segment	0
Chat app.office.chat	office	An app inside the segment	1
Meet app.office.meet	office	An app inside the segment	0
Sign app.office.sign	office	An app inside the segment	0
Browser app.office.browser	office	An app inside the segment	1
PlexiDocs app.office.docs	office	An app inside the segment	0
PlexiSheets app.office.sheets	office	An app inside the segment	0
PlexiSlides app.office.slides	office	An app inside the segment	0
PlexiDiagrams app.office.diagrams	office	An app inside the segment	0
PlexiDesign app.office.design	office	An app inside the segment	0
PlexiDraw app.office.draw	office	An app inside the segment	0

Every destination the app can navigate to 45

Read from the View union (stores/view.ts).

NAME	MODULE	WHAT IT IS	TEST MENTIONS
home view.home	Navigation	No parameters — a place, not a thing	13
all-tasks view.all-tasks	Navigation	No parameters — a place, not a thing	0
rooms view.rooms	Navigation	No parameters — a place, not a thing	2
desks view.desks	Navigation	Carries: roomId	3
shared view.shared	Navigation	No parameters — a place, not a thing	3
trash view.trash	Navigation	No parameters — a place, not a thing	7
attention view.attention	Navigation	No parameters — a place, not a thing	8
calendar view.calendar	Navigation	No parameters — a place, not a thing	18
project-dashboard view.project-dashboard	Navigation	Carries: projectId	4
task view.task	Navigation	Carries: taskId	301
connected-app view.connected-app	Navigation	Carries: appId	2
vault view.vault	Navigation	No parameters — a place, not a thing	2
messages view.messages	Navigation	No parameters — a place, not a thing	7
inbox view.inbox	Navigation	No parameters — a place, not a thing	5
mail view.mail	Navigation	No parameters — a place, not a thing	17
documents view.documents	Navigation	No parameters — a place, not a thing	10
office view.office	Navigation	Carries: app, doc	13
plexidesk view.plexidesk	Navigation	Carries: app	3
plexipeople view.plexipeople	Navigation	Carries: app	2
plexibrain view.plexibrain	Navigation	Carries: app	5
plexii view.plexii	Navigation	No parameters — a place, not a thing	5
design view.design	Navigation	No parameters — a place, not a thing	17
draw view.draw	Navigation	No parameters — a place, not a thing	8
document view.document	Navigation	Carries: documentId	29
livedoc view.livedoc	Navigation	Carries: liveDocId	1
livefolder view.livefolder	Navigation	Carries: liveFolderId	1
collaborations view.collaborations	Navigation	No parameters — a place, not a thing	0
insights view.insights	Navigation	No parameters — a place, not a thing	1
files view.files	Navigation	No parameters — a place, not a thing	11
organization view.organization	Navigation	No parameters — a place, not a thing	2
people-map view.people-map	Navigation	No parameters — a place, not a thing	0
suite view.suite	Navigation	No parameters — a place, not a thing	4
product view.product	Navigation	Carries: productKey	0
knowledge view.knowledge	Navigation	Carries: entryId	13
meetings view.meetings	Navigation	No parameters — a place, not a thing	1
apps view.apps	Navigation	No parameters — a place, not a thing	4
forms view.forms	Navigation	No parameters — a place, not a thing	2
sign view.sign	Navigation	No parameters — a place, not a thing	3
search view.search	Navigation	No parameters — a place, not a thing	3
projects view.projects	Navigation	No parameters — a place, not a thing	12
reports view.reports	Navigation	No parameters — a place, not a thing	7
flows view.flows	Navigation	No parameters — a place, not a thing	4
api view.api	Navigation	No parameters — a place, not a thing	6
marketplace view.marketplace	Navigation	No parameters — a place, not a thing	0
mdext view.mdext	Navigation	Carries: path	0

Building blocks you place on a desk 55

Read from the widget catalogue, including its written hint.

NAME	MODULE	WHAT IT IS	TEST MENTIONS
Tasks widget.task-list	Desk canvas / Tools	This desk's own tasks, with the whole workspace one click away	6
Metrics widget.metrics	Desk canvas / Tools	Several numbers read together, each with its own history	4
Contacts widget.contacts	Desk canvas / Comms	The handful of people this desk is about	3
Inbox widget.inbox	Desk canvas / Comms	Your mail, narrowed to what this desk is about by a rule you write	5
Map widget.location-map	Desk canvas / Files	A real address on a real map — search, pan and zoom (OpenStreetMap)	3
Calendar widget.calendar	Desk canvas / Tools	The month, marked with this desk's due dates and booked time	18
Stat card widget.stat-card	Desk canvas / Tools	A number worth watching, and how it got there	5
Gallery widget.gallery	Desk canvas / Files	A grid of pictures, kept in this workspace	2
Attention widget.attention	Desk canvas / Tools	This desk's attention items — or all of them	8
Custom widget.custom	Desk canvas / Tools	Describe any tool you need and have it built here	6
Image widget.image-gen	Desk canvas / Tools	Describe an image and generate it here	3
Sticky widget.sticky	Desk canvas / Notes	Small colored note pinned to the desk	148
Note widget.note	Desk canvas / Notes	Larger paper for longer writing	78
Markdown widget.markdown	Desk canvas / Notes	One portable note in markdown — slash menu, export, copy out. Use Page for rich multi-block docs.	17
App launcher widget.local-app-launcher	Desk canvas / Layout	One-click tile that launches a native Mac app — drag a Local app from the sidebar to create one	3
File or link widget.file	Desk canvas / Files	Upload any file (PDF, image, video, audio, doc) or paste a cloud link (Google Doc, Drive, Notion). Auto-detects type and renders the right preview.	27
Drive (folder) widget.drive	Desk canvas / Files	A folder from your Files, pinned to this desk. Lists its contents, opens it in Files, and any file you drop or add saves straight into that folder.	2
Field widget.field	Desk canvas / Tools	A single typed field — text, number, select, checkbox, attachment, button. Pick the type after creating.	13
Page widget.page	Desk canvas / Notes	A rich multi-block document (headings, lists, todos, slash-command AI). Use Markdown for a single portable note.	22
Table widget.table	Desk canvas / Tools	Airtable-style database — rows + columns of typed fields	64
Chart widget.chart	Desk canvas / Tools	PlexiDash — bar, line, area, pie or KPI charts from a table. Several on a desk make a dashboard.	11
Document widget.doc	Desk canvas / Files	A Word-class document. Create new, open a .docx, or place an existing one.	103
Spreadsheet widget.sheet	Desk canvas / Files	An Excel-class spreadsheet with formulas. Create new, import .xlsx/csv, or place an existing one.	38
Slides widget.slides	Desk canvas / Files	A PowerPoint-class deck. Create new, import .pptx, or place an existing one.	26
Flowchart widget.map	Desk canvas / Files	Flowcharts, process maps, org charts and mind maps (PlexiDiagrams) — diagram by hand or generate with AI.	15
Design widget.design	Desk canvas / Files	A PlexiDesign layout — brochures, posters, newsletters and one-pagers. Master pages, threaded text frames, guides and print-ready output.	17
Artwork widget.draw	Desk canvas / Files	A PlexiDraw artwork — vector shapes and bezier paths alongside painted layers, with pathfinder booleans, brushes and blend modes.	8
SpeedDeck widget.streamdeck	Desk canvas / Tools	Elgato-style 10x3 macro pad — launch apps, run macros, control media, organise with folders. Toggle between this-task and Universal scopes.	3
Task link widget.task-link	Desk canvas / Layout	Reference another task on this canvas — drag a task from the sidebar onto the canvas to create one	3
Browser widget.webview	Desk canvas / Web	Any URL — a focused browser tab for this task	29
PDF widget.pdf	Desk canvas / Files	PDF document via URL (<i>legacy (folded into File)</i>)	12
Doc widget.gdoc	Desk canvas / Files	Google Docs (<i>legacy (folded into File)</i>)	4
Sheet widget.gsheets	Desk canvas / Files	Google Sheets (<i>legacy (folded into File)</i>)	4
Slides widget.gslide	Desk canvas / Files	Google Slides (<i>legacy (folded into File)</i>)	4
Chat widget.chat-thread	Desk canvas / Comms	A live chat panel bound to a channel — read and reply without leaving the desk	5
Email widget.email	Desk canvas / Comms	Gmail / Outlook inbox view (<i>legacy (folded into File)</i>)	4
Image widget.image	Desk canvas / Files	Image from URL (<i>legacy (folded into File)</i>)	23
Video widget.video	Desk canvas / Files	Video from URL (mp4 / webm) (<i>legacy (folded into File)</i>)	11
Calc widget.calculator	Desk canvas / Tools	Small inline calculator	10
Color widget.color	Desk canvas / Tools	Color picker + screen eyedropper	12
Timer widget.timer	Desk canvas / Tools	Countdown timer with audio cues — beeps as time runs out	14
Section widget.section	Desk canvas / Layout	A labeled frame to group widgets visually — renders behind everything	25
Minimap widget.minimap	Desk canvas / Layout	Bird's-eye view of the canvas — drag the viewport rect to pan, click anywhere to jump there (<i>legacy (folded into File)</i>)	13
Voice note widget.voice-recorder	Desk canvas / Tools	Record a voice note — AI transcribes (full / cleaned / summary), suggests tasks and widgets, then sends results anywhere on the canvas	7
Mind map widget.mindmap	Desk canvas / Tools	Explore an idea — AI suggests branches, tasks, and the Agent OS agents that can execute on each node	18
Diagram widget.diagram	Desk canvas / Tools	Flowcharts, hierarchies, server/software design, Venn — boxes, circles, text & image nodes joined with connectors	11
Scratchpad widget.scratchpad	Desk canvas / Tools	Freeform sketch surface — draw, annotate and think visually with pressure-sensitive ink	4
Shape widget.shape	Desk canvas / Layout	A vector shape — rectangle, ellipse, diamond, triangle, hexagon, star, line or arrow — with fill, stroke and an optional label	15
Card widget.card	Desk canvas / Notes	A titled callout card with an accent bar, bold title and body	12
Custom block widget.custom-block	Desk canvas / Tools	Design your own form/record — drop typed fields and place them where you want; save as a reusable template	4
Desk agent widget.agent	Desk canvas / Tools	A standing AI agent. Wire widgets INTO it as inputs, give it an instruction and a trigger, and it works on its own	27
Send to a URL widget.webhook	Desk canvas / Tools	An outbound webhook. Wire a tool into it and its content is POSTed to your URL whenever it changes — send desk data out to Slack, Zapier or any HTTP endpoint	5
Receive from a URL widget.inbound-hook	Desk canvas / Tools	An inbound webhook. Gives you a unique URL — when an external system (Stripe, a form, curl) POSTs to it, the payload lands here and fires any wire drawn OUT of this tool	4
Portal widget.portal	Desk canvas / Layout	A live window into another task's desk — watch it at a glance and click to dive in	6
Living Doc widget.living-doc	Desk canvas / Notes	A doc that writes itself — give it a brief and it keeps a living summary of this desk	9

The IPC surface, grouped by namespace 100

Counted from ipcMain.handle registrations in main.

NAME	MODULE	WHAT IT IS	OPS	TEST MENTIONS
fileManager ipc.fileManager	Main process	30 operations: importFolder, list, get, path, createFolder, rename, move, moveToOrg...	30	3
workspace ipc.workspace	Main process	27 operations: exportJson, importJson, ask, askStream, related, pending, advanceBaseRev, markPushed...	27	12
ai ipc.ai	Main process	24 operations: dailyBrief, suggestPageContent, routeCommand, suggestDocContent, rewriteSelection, suggestSheetColumns, suggestFormula, fillSheetRange...	24	15
workItems ipc.workItems	Main process	23 operations: list, get, create, updateFields, setState, reclassify, snooze, markRead...	23	6
meetings ipc.meetings	Main process	19 operations: list, get, create, saveSegments, saveAudioTakes, audioInfo, loadAudioTakes, keepAudio...	19	2
documents ipc.documents	Main process	18 operations: generateSlides, list, get, create, update, delete, listTrashed, restore...	18	17
settings ipc.settings	Main process	18 operations: encryptionAvailable, hintAnthropic, hintOpenAI, saveAnthropicKey, saveOpenAIKey, clearAnthropicKey, clearOpenAIKey, hintTenor...	18	2
extcal ipc.extcal	Main process	16 operations: list, addInternal, setDefaultInternal, pushNow, listEvents, add, update, remove...	16	0
mail ipc.mail	Main process	16 operations: getAccount, saveAccount, testAccount, clearAccount, list, get, archive, listCategories...	16	47
nodes ipc.nodes	Main process	15 operations: staleDesks, list, get, create, update, listTrash, restoreTree, delete...	15	41
files ipc.files	Main process	13 operations: ingestPath, ingestBuffer, get, ingestUrl, internaliseDeskImages, delete, extractText, read...	13	17
projects ipc.projects	Main process	13 operations: list, plan, setTaskPlan, addDep, setDep, removeDep, reschedule, captureBaseline...	13	4
agents ipc.agents	Main process	12 operations: run, designProfile, previewInput, getWorkspaceOverride, setWorkspaceOverride, revealInFinder, create, invoke...	12	3
vault ipc.vault	Main process	12 operations: meta, isUnlocked, create, unlock, lock, listEntries, createEntry, updateEntry...	12	1
shares ipc.shares	Main process	11 operations: listAll, listForEntity, create, revoke, setScope, delete, inbox, accept...	11	2
tables ipc.tables	Main process	11 operations: list, get, create, update, delete, listRows, createRow, updateRow...	11	8
widgets ipc.widgets	Main process	10 operations: get, listByTask, listByKind, create, createOptional, update, delete, restore...	10	40
customWidget ipc.customWidget	Main process	10 operations: generate, name, list, get, save, delete, rename, markUsed...	10	0
aiChat ipc.aiChat	Main process	10 operations: listConversations, getConversation, createConversation, setConversationMode, setConversationWebSearch, appendMessage, setMessageApplied, renameConversation...	10	4
liveDesk ipc.liveDesk	Main process	9 operations: get, list, start, publish, queuePublish, uploadAsset, setPaused, stop...	9	1
sign ipc.sign	Main process	9 operations: list, get, create, update, delete, send, sign, decline...	9	10
design ipc.design	Main process	8 operations: generateImage, export, planLayout, searchPhotos, fetchImage, removeBackground, generateContent, generateVariations	8	8
mailTags ipc.mailTags	Main process	8 operations: list, listForNode, create, update, remove, pin, exclude, reorder	8	1
contacts ipc.contacts	Main process	8 operations: list, listForNode, create, update, remove, link, unlink, desks	8	0
knowledge ipc.knowledge	Main process	8 operations: list, get, search, create, update, delete, reindex, semanticActive	8	3
connectedApps ipc.connectedApps	Main process	7 operations: list, create, update, delete, reorder, touch, findByHostname	7	0
reports ipc.reports	Main process	7 operations: list, get, create, update, delete, generate, runDue	7	1
flows ipc.flows	Main process	7 operations: list, get, create, update, delete, run, runDue	7	0
streamdeck ipc.streamdeck	Main process	6 operations: execute, openAccessibilitySettings, checkAccessibility, promptAccessibility, openSettingsApp, revealApplnFinder	6	1
api ipc.api	Main process	6 operations: status, setEnabled, setPort, listTokens, createToken, revokeToken	6	310
signals ipc.signals	Main process	5 operations: record, list, matchState, markPrompted, outcome	5	2
decisions ipc.decisions	Main process	5 operations: create, list, forObject, withRisk, cancel	5	6
account ipc.account	Main process	5 operations: load, saveSession, clearSession, setSkipped, setCachedEmail	5	20
snapshots ipc.snapshots	Main process	5 operations: create, list, get, restore, branch	5	1
chat ipc.chat	Main process	5 operations: send, cancelStream, sendStream, hasApiKey, proactiveWelcome	5	16
crdt ipc.crdt	Main process	5 operations: record, unsynced, markSynced, knowlIds, eventsForObject	5	2
localApp ipc.localApp	Main process	5 operations: pick, describe, launch, isRunning, refreshIcon	5	0
browserAgent ipc.browserAgent	Main process	5 operations: start, stop, steer, consent, deliver	5	0
apps ipc.apps	Main process	5 operations: list, get, create, update, delete	5	2
forms ipc.forms	Main process	5 operations: list, get, create, update, delete	5	0
activity ipc.activity	Main process	5 operations: getState, setEnabled, read, logPress, wipe	5	8
context ipc.context	Main process	4 operations: related, health, markReviewed, resumeSummary	4	15
widgetLinks ipc.widgetLinks	Main process	4 operations: listByTask, create, update, delete	4	2
memory ipc.memory	Main process	4 operations: list, remember, forget, extractDocuments	4	40
office ipc.office	Main process	4 operations: importDocx, exportDocx, exportPdf, pickImage	4	10
crash ipc.crash	Main process	4 operations: report, list, unforwarded, markForwarded	4	2
timeblocks ipc.timeblocks	Main process	4 operations: list, create, update, delete	4	2
backup ipc.backup	Main process	4 operations: info, export, restore, revealFolder	4	0
voice ipc.voice	Main process	4 operations: getProvider, preloadLocal, transcribeLive, setProvider	4	1
agentBrowser ipc.agentBrowser	Main process	4 operations: createRun, stopRun, endRun, perform	4	2
fileImport ipc.fileImport	Main process	4 operations: pick, run, pickGrid, parseGrid	4	2
wireRuns ipc.wireRuns	Main process	3 operations: record, listByWire, listByTask	3	0
metrics ipc.metrics	Main process	3 operations: get, webContents, buildBinding	3	3
templates ipc.templates	Main process	3 operations: list, createFromTask, delete	3	0
sheet ipc.sheet	Main process	3 operations: import, export, runMacro	3	9
focus ipc.focus	Main process	3 operations: start, complete, recent	3	14
clusters ipc.clusters	Main process	3 operations: list, save, delete	3	0
trail ipc.trail	Main process	3 operations: record, recent, summarize	3	1
dashboard ipc.dashboard	Main process	3 operations: getLayout, setLayout, resetLayout	3	2
energy ipc.energy	Main process	3 operations: log, current, recent	3	1
mindmap ipc.mindmap	Main process	3 operations: expand, listAgents, suggestAgents	3	0
app ipc.app	Main process	3 operations: setZoomFactor, isPreviewBuild, setBackgroundColor	3	84

NAME	MODULE	WHAT IT IS	OPS	TEST MENTIONS
docComments ipc.docComments	Main process	3 operations: list, add, resolve	3	0
deskLayout ipc.deskLayout	Main process	2 operations: load, save	2	2
speeddeck ipc.speeddeck	Main process	2 operations: loadUniversal, saveUniversal	2	0
agent ipc.agent	Main process	2 operations: step, verify	2	6
setup ipc.setup	Main process	2 operations: suggest, buildFromPrompt	2	3
map ipc.map	Main process	2 operations: export, import	2	7
brand ipc.brand	Main process	2 operations: get, set	2	3
slides ipc.slides	Main process	2 operations: export, import	2	15
history ipc.history	Main process	2 operations: record, recent	2	3
model ipc.model	Main process	2 operations: get, set	2	14
session ipc.session	Main process	2 operations: getActiveOrg, setActiveOrg	2	5
haptics ipc.haptics	Main process	2 operations: available, fire	2	0
export ipc.export	Main process	2 operations: html, pdf	2	3
brain ipc.brain	Main process	2 operations: ingestWorkspace, coverage	2	3
voiceCommand ipc.voiceCommand	Main process	2 operations: getPrefs, setPrefs	2	0
browserConsent ipc.browserConsent	Main process	2 operations: list, revoke	2	0
media ipc.media	Main process	2 operations: micStatus, askMic	2	2
mdext ipc.mdext	Main process	2 operations: read, write	2	1
planner ipc.planner	Main process	1 operation: selectItems	1	0
notifications ipc.notifications	Main process	1 operation: post	1	0
webhooks ipc.webhooks	Main process	1 operation: send	1	1
wires ipc.wires	Main process	1 operation: runTransform	1	2
assistant ipc.assistant	Main process	1 operation: standup	1	2
calendar ipc.calendar	Main process	1 operation: addMeetingIcs	1	3
resume ipc.resume	Main process	1 operation: generate	1	1
livingPage ipc.livingPage	Main process	1 operation: regenerate	1	1
image ipc.image	Main process	1 operation: generateToFile	1	2
draw ipc.draw	Main process	1 operation: export	1	4
telemetry ipc.telemetry	Main process	1 operation: collect	1	1
onboarding ipc.onboarding	Main process	1 operation: record	1	1
bodyDouble ipc.bodyDouble	Main process	1 operation: tick	1	0
smartStack ipc.smartStack	Main process	1 operation: propose	1	0
dashboardWizard ipc.dashboardWizard	Main process	1 operation: refine	1	0
update ipc.update	Main process	1 operation: check	1	24
search ipc.search	Main process	1 operation: query	1	3
people ipc.people	Main process	1 operation: setDirectory	1	3
marketplace ipc.marketplace	Main process	1 operation: apply	1	0
gif ipc.gif	Main process	1 operation: search	1	1

Renderer stores 71

Every store module, with how many files import it.

NAME	MODULE	WHAT IT IS	DEPS	LOC	TEST MENTIONS
nodes store.nodes	Renderer state	A "desk" (marketing) / "project" (sidebar) is a top-level folder. The multiple_desks capability caps how many a tier may have.	129	342	333
view store.view	Renderer state	Current pane view — replaces the assumption that the main area always shows a task canvas. As PlexiDesk grows into a workspace OS, the main pane routes between Home Dashboard, All Tasks, per	124	281	391
widgets store.widgets	Renderer state	Re-export the Focus-Mode nav helpers so consumers (WidgetFocusMode, the split surfaces) can import them from the store alongside useWidgetStore.	121	748	284
account store.account	Renderer state	Account store — single source of truth for "am I signed in, and as who?" on the renderer. Lifecycle: 1. App boots → init() loads the cached session from main (encrypted via safeStorage). If	69	251	140
documents store.documents	Renderer state	Documents store — the standalone office files (doc / sheet / slides). Holds the list for the hub and the one open document for the editor. Body edits are debounced to disk so typing stays sm	48	293	124
workItems store.workItems	Renderer state	The work_item store (Attention S3, \$4). Work items NEVER pass through useNodeStore — listNodes excludes the kind, and every desk/room surface is blind to them by construction.	45	107	21
capabilities store.capabilities	Renderer state	Capability store — caches the per-tier resolved capability map for the signed-in user, refreshed on auth change + window focus + an imperative `refresh()` call. Consumers read via `useCapabi	24	304	16
org store.org	Renderer state	The active organisation is the tenancy boundary for the whole workspace. The user's own local-first data lives in a reserved "Personal" org that is always available offline; the other option	22	123	132
messaging store.messaging	Renderer state	Apply a reaction add/remove to a message in the conversation map, immutably and idempotently (so the actor's own optimistic update plus the echoed broadcast do not double count). Returns a n	21	732	22
tables store.tables	Renderer state		21	207	71
actionHistory store.actionHistory	Renderer state	One unified undo/redo timeline for structural workspace actions — creating, deleting, moving, renaming, resizing, recolouring, linking tasks, folders and widgets.	20	142	3
focusSession store.focusSession	Renderer state	Lightweight store for the "currently running" 5-min-Promise / focus session. History is read from the DB on demand (Garden, Trail later); this store holds only the active session + a per-tic	19	230	4
links store.links	Renderer state	Spatial-link store. Mirrors the widgets store's per-task lifecycle: loadForTask wipes + fetches; clear empties on task switch. Optimistic updates so the SVG overlay redraws immediately witho	19	133	60
timeBlocks store.timeBlocks	Renderer state	Calendar time blocks for the currently-viewed range. The view sets the range (a week / a day); the store loads the blocks that overlap it and keeps an in-memory copy that create/update/remov	18	126	19
meetings store.meetings	Renderer state	PlexiMeet store. Thin wrapper over the main-process meetings store via IPC. Reads only real meetings; an empty history shows an honest empty state.	16	37	32
presence store.presence	Renderer state	Account-level presence (the "People Map"): who across your org/team is online right now and what they're doing. Rides the same authenticated messaging socket.	16	198	32
chat store.chat	Renderer state		14	1176	117
files store.files	Renderer state	Object-URL cache so we can render the same file in multiple widgets without re-reading the bytes per render. Cleared on file delete.	14	140	96
knowledge store.knowledge	Renderer state	PlexiBrain knowledge base store. Thin wrapper over the main-process store via IPC; the renderer holds the current list and writes through. Reads only real entries, never seeds samples, so an	14	38	46
upgradePrompt store.upgradePrompt	Renderer state	Global "upgrade to unlock" prompt. Any gated action (a locked widget tile, hitting the desk limit, a Pro-only feature) calls promptUpgrade() with a short reason; the single <UpgradePrompt	13	29	0
connectedApps store.connectedApps	Renderer state		12	92	4
contextHealth store.contextHealth	Renderer state	Renderer-side Context Health (plexi-4.0). Reads the live context.* endpoints so desks can surface "what changed since you last looked" and "which related desks need attention".	12	119	7
assistantChrome store.assistantChrome	Renderer state	Chrome state for the global assistant: whether it is open, which of the three display modes it is in, and the sidebar-mode dock width.	11	148	3
shares store.shares	Renderer state	ShareService singleton — null in local-mock dev mode (no server), instantiated in prod / when explicitly configured. Created lazily so the renderer doesn't open a connection until the user a	10	265	16
webPanel store.webPanel	Renderer state	The in-app browser panel's state (A2, AI-03, R4/R13): one right-panel browser serves citations, omnibar URLs, and web search results — the web never leaves Plexii; the system browser is an e	10	87	0
quickCreate store.quickCreate	Renderer state	Global quick-create: lets the command palette (Cmd+K) ask any module to create a new item, from anywhere in the app, without the user first navigating to that module and hunting for its "New	9	29	1
templates store.templates	Renderer state		9	44	12
fileManager store.fileManager	Renderer state	State for the file/folder manager view. The current folder (cwd, null = root) drives the listing; view mode and sort are persisted so the manager opens the way the user left it.	8	428	9
vault store.vault	Renderer state		7	148	11
call store.call	Renderer state	PlexiCam: peer-to-peer live audio/video calls. The signal server only relays the SDP offer/answer and ICE candidates between two accounts in the same presence audience; the media itself flow	6	442	329
energy store.energy	Renderer state		6	81	1
forms store.forms	Renderer state	PlexiForms store. A form is a thin record over a backing table: creating a form creates the table that holds its fields (columns) and responses (rows). Reads only real forms; an empty worksp	6	48	23
mail store.mail	Renderer state	Mail store — the IMAP inbox the user connects with their own mailbox. All the IMAP work happens in the main process; this store holds the account status, the message list (envelope only), an	6	380	140
mailTags store.mailTags	Renderer state	The user's mail tags. Held apart from the mail store because a tag outlives any particular mailbox fetch: tags are the user's own structure and survive a refresh, a reconnect, or having no m	6	152	3
notice store.notice	Renderer state	DEC-091 — the house NOTICE toast: one transient, non-blocking line for "a thing happened somewhere you are not looking", with an optional action that takes you there.	6	40	21
onboarding store.onboarding	Renderer state	Modular onboarding. There is one always-available "core" first-run flow plus any number of short, replayable feature tours.	6	209	34
apps store.apps	Renderer state	PlexiBuild store. Thin wrapper over the main-process apps store via IPC. Reads only real apps; an empty workspace shows an honest empty state.	5	37	27
captureConsole store.captureConsole	Renderer state	The capture console's open/close state (Attention S5). Opened by the fb:command-new-work-item seam (palette, shortcuts, future surfaces) with optional prefill text; the console itself lives	5	46	2
completionOffer store.completionOffer	Renderer state	DEC-052 (Track D) — the offer surface's state. The contract, from the ruling, made structural: - NEVER auto-complete: the only writer is the toast's confirm, and it runs the same accounted c	5	69	2

NAME	MODULE	WHAT IT IS	DEPS	LOC	TEST MENTIONS
meetingRoom store.meetingRoom	Renderer state	PlexiMeet live rooms: a multi-party meeting built as a WebRTC mesh. Each member holds one peer connection to every other member; the signal server only relays the roster and the SDP/ICE, nev	5	810	5
signInPrompt store.signInPrompt	Renderer state	A tiny shared flag so anywhere in the app (the Settings account section, a capability-expired prompt, a "sign in to invite" affordance) can pop the sign-in modal on demand.	5	24	1
wrapup store.wrapup	Renderer state	End-of-conversation wrap-up. When a meeting or call ends with a recording, this drives the one honest pipeline: transcribe the mixed audio, ask the AI for a grounded summary and the delivera	5	457	13
guestCapture store.guestCapture	Renderer state	M6 (SPEC-003 P6, CR-12) — Guest Capture: record a meeting that happens OUTSIDE Plexii (Zoom, Meet, Teams — an external joinUrl booking, or any call playing through this machine).	4	184	3
sign store.sign	Renderer state	PlexiSign store. A thin client over the main-process signing engine. Reads only real requests; an empty workspace has none. All signing transitions and the completion certificate are compute	4	82	260
agentLoop store.agentLoop	Renderer state	State for one autonomous agent run: the live transcript the UI shows, the gated actions deferred for the user's approval, and a 'running' flag other surfaces read as a lock (ProposalCards di	3	53	0
brand store.brand	Renderer state	The organization Brand Kit, read by every surface that wants to present on brand (PlexiDesign today; Docs/Sheets/Slides/Projects as they adopt it).	3	34	31
calendars store.calendars	Renderer state	The calendar list, cached for colour lookups. Every surface that draws an entry needs to know which calendar it belongs to and what colour that calendar is.	3	65	5
dashboardLayouts store.dashboardLayouts	Renderer state	Per-dashboard layouts. Each dashboard has a list of card IDs in render order, a column count (1/2/3), and a per-card size map. Keyed by 'home' (master dashboard) or a project node id, so eve	3	136	1
deskView store.deskView	Renderer state	Per-desk view mode. A desk can be shown as the infinite spatial Canvas (the default) or as Columns (stacked scrollable walls).	3	72	2
editorCommands store.editorCommands	Renderer state	A registry the active rich editor (Document, Slides) publishes its own commands into, so the global command palette (Cmd+K) can drive the editor without the palette needing to know anything	3	68	0
focusClusters store.focusClusters	Renderer state	Renderer store for Focus-Mode CLUSTERS (split "groups"), the persisted per-desk counterpart to the live focusSplit store.	3	188	0
knock store.knock	Renderer state	PlexiPeople knock-to-connect. A knock is a lightweight "I want to reach you" ping between two people who can see each other.	3	90	3
overlay store.overlay	Renderer state	A registry of the floating menus and popovers that are open right now: the desk control pill's fly-out, the canvas context menu, and any other menu that opts in. Two parts of the app read it	3	61	70
syncStatus store.syncStatus	Renderer state	Honest, user-visible sync status. The audit (review finding M1/M2) noted that workspace/cloud sync failed completely invisibly: an offline blip self-healed (correct), but a persistent server	3	69	2
widgetSetup store.widgetSetup	Renderer state	Drives the per-widget AI setup preview. A menu action calls start(widgetId); the WidgetSetupPreview component (mounted once) runs the suggest IPC, shows the drafted items for approval, and a	3	21	2
aiAssistPreview store.aiAssistPreview	Renderer state	AI Assist preview store. Every AI Assist action opens a single preview before anything is written.	2	67	0
docCollab store.docCollab	Renderer state	Client state for the currently-open live document or live folder: its metadata and baseline body, fetched once on open.	2	53	2
externalEvents store.externalEvents	Renderer state	Events mirrored from Google / Outlook / an ICS feed. Read-only by construction: there is no setter here that writes back, because these rows belong to a calendar somewhere else.	2	40	0
liveFolderEntries store.liveFolderEntries	Renderer state	WS01 sync substrate — the materialised tree of a LIVE folder, per folder id. A live folder has no main-process table: its durable store is the CRDT change log on the signal server, replayed	2	95	0
mailModal store.mailModal	Renderer state	Which email is open, if any. A store rather than local state because the reader is opened from several places that do not know about each other -- the inbox widget on a desk, the mail view,	2	29	0
peerBodyDouble store.peerBodyDouble	Renderer state	Singleton matcher instance — created lazily on first use. Feature-flag chooses LocalMockMatcher (BroadcastChannel; two windows on the same machine can pair) vs RemoteMatcher (WebSocket to th	2	156	0
relatedDesks store.relatedDesks	Renderer state	Open/close state for the Related Desks manager. Any surface (the command palette, a desk action) opens it for a given desk; the modal mounted in App reads this store and renders itself.	2	19	0
speeddeck store.speeddeck	Renderer state	Universal SpeedDeck store. Every SpeedDeck widget in "Universal" scope reads from THIS store and writes back to it; the store mirrors itself to disk via the speeddeck:" IPC.	2	115	0
autonomy store.autonomy	Renderer state	The client home of the three-level autonomy policy. The user's system-wide and per-assistant choices live in per-account synced state (consistent across their devices); the org ceiling/defau	1	138	1
browserAgentRuns store.browserAgentRuns	Renderer state	Renderer state for agentic-browsing runs (A6/B2): the runtime's client. Main drives the loop and pushes events; this store is the one place the renderer accumulates them — B3's visible run (	1	249	0
featureVotes store.featureVotes	Renderer state	Upvotes for coming-soon and planned PlexiSuite products. Each vote is a real account voting for a real feature key, so the team can both rank demand and notify the exact people who asked onc	1	141	1
focusSplit store.focusSplit	Renderer state	Session-local split state for Focus Mode (spec: docs/FOCUS-SPLIT-VIEW-SPEC.md). The store holds the LIVE arrangement; the deterministic geometry all lives in splitGeometry.ts (unit-tested) —	1	143	1
meetingLaunch store.meetingLaunch	Renderer state	Drives the "start a meeting" dialog. When a meeting is launched from an artifact (a document, sheet, deck, drawing, design or desk) the dialog opens so the user can add attendees and choose	1	22	0
openTray store.openTray	Renderer state	What you have open, kept per device. This is window state, not workspace data: which things YOU have on this machine right now, the same way a browser's tabs are not part of the pages.	1	105	3
pinLayer store.pinLayer	Renderer state	The global pin layer store (spec §7). Pins are a local viewing aid that follows the user across navigation, persisted in localStorage. Dedupe is by (kind, refId) so pinning the same object t	1	81	1
radar store.radar	Renderer state	Holds the current radar suggestions + the per-kind accept counters that are the foundation for learned autonomy (a category the user keeps accepting can later be escalated toward hands-free)	1	93	3

Substantial main-process modules (AI, database, mail) 120

Files over 80 lines, with dependants and size.

NAME	MODULE	WHAT IT IS	CALLERS	DEPS	LO
anthropic engine.anthropic	Main / ai	Static imports (not lazy require): electron-vite only bundles the static import graph, so a runtime require("../db/documents") resolved against out/main/ (which has no db/ dir) and threw MOD	38	11	61k
workspaceSync engine.workspaceSync	Main / db	—— Park-inbound (§2.1 receiver defensiveness; GAP-013 repair) ———— A row this build cannot materialize (its kind fails the local CHECK — e.g.	12	3	11k
migrations engine.migrations	Main / db	Everything that turns an empty SQLite database into the current one: the declarative schema, then the accumulated migrations. This was inside getDb, which is where it had always lived and wh	4	2	98k
workItems engine.workItems	Main / db	work_item data model — columns, projection, satellites, remote apply (Attention layer S2; ARCHITECTURE §2.2–§2.4, §3). One code path (F008): every work_item write — user, AI, or sync arrival	11	45	91k
files engine.files	Main / db		33	14	89k
browserActions engine.browserActions	Main / ai	The deterministic action bridge (A6/B1, R27/R29). Every way the agent runtime can touch a page funnels through ONE door — performAgentAction — so the kill switch, the step ceiling, and the R	3	4	75k
imap engine.imap	Main / mail	IMAP client — the thin layer that actually talks to the user's mail server. PlexiDesk connects directly from the desktop using imapflow, lists the most recent INBOX messages (envelope only,	10	2	64k
nodes engine.nodes	Main / db		44	129	59k
browserAgent engine.browserAgent	Main / ai	The agentic-browsing loop (A6/B2, R26/R27/R28). One runtime, Plexii's:	68	1	57k
schema engine.schema	Main / db	The complete SQLite schema, as one idempotent script of CREATE TABLE IF NOT EXISTS / CREATE INDEX IF NOT EXISTS statements.	—	1	55k
voiceNote engine.voiceNote	Main / ai	Voice-note AI pipeline. Three stages, each callable independently from the renderer: 1. transcribeAudio(audioBytes, mimeType) → POST to OpenAI /v1/audio/transcriptions (model "whisper-1"), r	4	2	55k
projectPlan engine.projectPlan	Main / db	The PlexiProjects plan store. A project is an existing folder node; its plan is the task nodes beneath it plus the finish-to-start dependencies between them.	7	2	53k
mindMap engine.mindMap	Main / ai	Mind-mapper AI pipeline. Three independently-invokable helpers: 1. expandMindMapNode(rootPath, nodeContext) → Claude reads the path from the root to the selected node (so it has full themati	1	1	47k
agentDispatcher engine.agentDispatcher	Main / ai	Agent dispatcher — Phase 2C FOUNDATION. This is NOT the autonomous agent runtime. That's a 2-4 week project in its own right (PHASE_3 markers throughout, see below).	2	1	46k
deskBundle engine.deskBundle	Main / db	One desk, packed into a file that can travel. workspaceExport.ts writes the WHOLE workspace and leaves file bytes on disk, which is right for "let me leave with everything" and wrong for "he	6	2	42k
aiChat engine.aiChat	Main / db	Local persistence for the AI-assistant chat: free-standing conversations and their messages, mirroring the documents.ts pattern (Row interfaces, rowTo mappers, prepared statements, org scopi	16	3	41k
mentionResolver engine.mentionResolver	Main / ai	Turn the user's @-mentions into real text (Phase 4.2). This is the piece that makes mentions genuinely better than click-to-pin.	2	3	39k
vault engine.vault	Main / db	—— Crypto constants ———— PBKDF2-SHA256, 600k iterations = OWASP 2023 recommendation. Increase over time.	13	7	39k
customWidget engine.customWidget	Main / ai	Custom widget generation -- the user describes a tool in plain language and the model writes it. What comes back is a self-contained HTML fragment: markup, styles and logic in one piece, wit	1	1	39k
intentRules engine.intentRules	Main / ai	Deterministic intent rules (Attention S5; ARCHITECTURE §6 + R011). The capture classifier's FIRST pass: hard triggers that resolve without any model call — most captures ("remind me to...", "f	2	2	37k
agentHistory engine.agentHistory	Main / ai	Agent invocation history + outcome bookkeeping. Three responsibilities: 1. Record every agent invocation as it happens, so the user can see what they ran, when, against which node, and what	9	2	36k
streamingEnvelope engine.streamingEnvelope	Main / ai	Incremental scanner for the model's { "reply": ..., "-items-": [...] } envelope. Both AI surfaces that stream ask Claude for the same shape — a reply string plus an array of objects — but t	1	2	36k
widgets engine.widgets	Main / db		43	121	36k
externalCalendars engine.externalCalendars	Main / db	Calendars that live somewhere else. External events are kept in their OWN table rather than folded into time_blocks, and that separation is the whole design.	13	4	35k
eventStore engine.eventStore	Main / db	Append-only Event Store (spec §35, §49, ADR-0001). The system of record for Plexii 4.0. Events are immutable and append-only, enforced by SQLite triggers, not convention (PLX-EVT-010, PLX-EV	2	23	35k
relationshipStore engine.relationshipStore	Main / db	Relationship store (spec §36, REQ-GPH) — provenance-carrying edges of the knowledge graph. Every edge is written with evidence (rejected at write time otherwise, PLX-GPH-001), a discovery me	2	2	34k
flows engine.flows	Main / db	PlexiFlow engine and store. A flow is a trigger plus an ordered list of actions the workspace already knows how to perform.	6	6	33k
search engine.search	Main / db	Global "find anything" search across the local workspace: node titles + descriptions, widget text, document bodies, table-row cells, file names, and PlexiBrain knowledge.	3	1	33k
nodeLifecycle engine.nodeLifecycle	Main / db	Node lifecycle mechanics — the single owner of trash/restore/purge/detach semantics for the nodes table (Attention layer S1; ARCHITECTURE §2.5).	8	4	32k
tables engine.tables	Main / db	—— fb_tables ————	36	21	29k
creditMode engine.creditMode	Main / ai	Credit mode — routing AI calls through the PlexiDesk signal server on our Anthropic key, billed against the account's prepaid balance, with automatic fall-back to the user's own key (BYOK) w	7	4	28k
workspaceExport engine.workspaceExport	Main / db	Portable workspace export/import. A .fbbackup (see backup.ts) is a SQLite copy: the right tool for "restore yesterday's workspace", and the wrong one for "let me leave". It needs this app's	1	1	28k
documents engine.documents	Main / db	Office documents store — CRUD for standalone doc / sheet / slides files. The body is persisted as a JSON string; the shape depends on doc_type and is validated only loosely here (the editors	18	48	27k
decisionStore engine.decisionStore	Main / db	Decision store (spec §37, REQ-DOM/REQ-APP) — persists Decisions with human-only ownership and approval (PLX-DOM-040), advisory-only AI commentary (PLX-DOM-041), the APP-020 approval gate, pe	3	1	25k
reports engine.reports	Main / db	PlexiReports store. A report is a saved selection of tables plus a schedule and recipients. Generating one reads the real table data, builds an honest data summary, and (when an Anthropic ke	5	4	24k
timeBlocks engine.timeBlocks	Main / db		8	18	24k
browserAgentEnvelope engine.browserAgentEnvelope	Main / ai	The browser-agent round envelope (A6/B2): pure parsing + sanitising for what the model returns each round, kept free of Electron/SDK deps so it unit-tests directly (the agentEnvelope.ts prec	2	2	23k
widgetInputs engine.widgetInputs	Main / db	What a custom widget can see, resolved from the wires drawn into it. The whole access model in one sentence: a generated widget reads exactly the widgets the user connected to it, and has no	2	2	23k
orchestrator engine.orchestrator	Main / ai	AI Orchestrator (spec §55, §67, §70, REQ-AI). The single internal abstraction through which all model invocation flows (AI-001): capability-aware routing that refuses under-capable models (A	2	2	23k
connectedApps engine.connectedApps	Main / db		2	12	22k
activity engine.activity	Main / db		5	5	22k
sign engine.sign	Main / db	—— fb_sign_requests (PlexiSign) ———— One row per signature request. signers and audit are JSON arrays — a request is a self-contained envelope, so th	1	4	22k
backup engine.backup	Main / db	Backup / export / restore for the single-file SQLite database. "Backup" here is a transactionally-consistent online snapshot via better-sqlite3's backup() — safe to take while the app is run	2	2	21k

NAME	MODULE	WHAT IT IS	CALLERS	DEPS	LO
account engine.account	Main / db	Account session persistence — main-process only. We keep two facts on disk: 1. The session token issued by the signal server's /accounts/login (or /accounts/signup).	15	69	214
agentBuilder engine.agentBuilder	Main / ai	Agent creation — writes a new agent markdown file into `<workspace>/claude/agents<slug>.md` following the kit's conventions. Bootstraps via Claude Sonnet to draft the body from a sho	2	1	210
mailTags engine.mailTags	Main / db	Where a user's mail tags live. A tag is a saved criterion, not a drawer (see the note on MailTag in shared/types.ts).	3	6	209
contacts engine.contacts	Main / db	The people a workspace deals with. This is the table that did not exist. Org members live on the signal server and are fetched per session; everybody else -- a conveyancer, a photographer, a	3	2	209
shares engine.shares	Main / db		5	10	209
retention engine.retention	Main / db	DEC-056 / DEC-057 — the retention sweep, and the caller that makes caps real. Every prune function in this codebase was written and then never called.	2	1	207
markdownToTiptap engine.markdownToTiptap	Main / ai	Minimal markdown -> Tiptap doc JSON converter for living-page output. Why hand-rolled: tiptap-markdown lives in the renderer (it's a Tiptap extension that mounts onto an editor instance).	2	1	198
eventSchemas engine.eventSchemas	Main / db	Event JSON-Schema registry and validator (spec \$64, PLX-EVT-043/044). Every Event type has a published, versioned schema at a stable dataschema URI, and events are validated against it.	6	1	188
memory engine.memory	Main / db	The self-building memory store (fb_memory). Durable facts / preferences / commitments about the user and their work, injected into the assistant's context so it stops starting cold.	4	4	188
mailTriage engine.mailTriage	Main / ai	Reading an inbox and proposing what to do with it. The model gets headers only — sender, subject, date, whether it was read, whether the sender published an unsubscribe.	1	2	184
canvasSnapshots engine.canvasSnapshots	Main / db	Desk time-travel snapshots. A snapshot is the full widget set for a task at a moment in time. We keep the last MAX_PER_TASK and prune older ones.	6	2	184
knowledge engine.knowledge	Main / db	—— fb_knowledge (PlexiBrain) —— CRUD plus a small relevance search used by both the PlexiBrain view and the AI grounding path. Mirrors the fb_ta	9	14	184
crashLog engine.crashLog	Main / db	Local crash log (WS03 reliability). The app already isolates a failing widget (WidgetErrorBoundary) and catches a fatal render error (ErrorBoundary), but the error itself only ever reached t	5	2	183
chatJson engine.chatJson	Main / ai	Pure string helpers for reading the model's { "reply": ..., "actions": [...] } chat envelope.	9	7	179
dashboardLayouts engine.dashboardLayouts	Main / db		2	3	176
actionLabel engine.actionLabel	Main / ai	One human line per action the assistant is preparing, for the retrieval trace. The trace's job is to let you watch the work happen — "∞ Email draft → Ryan", not "compose-mail".	1	1	169
telemetry engine.telemetry	Main / db	Local usage telemetry. Reads aggregate counts from this machine's database (widgets by kind, tasks, folders, focus minutes) plus a cumulative AI-call counter, and shapes them into a snapshot	13	9	169
customWidgets engine.customWidgets	Main / db	The user's personal library of AI-built widgets. A saved widget is a name, the plain-language spec that produced it, and a self-contained HTML document.	1	1	168
mailAccount engine.mailAccount	Main / mail	Mail account config — the IMAP connection details for the user's own mailbox. PlexiDesk talks to the mail server directly from the desktop, so the credentials live locally and never touch th	217	5	167
agents engine.agents	Main / ai	Agent governance (spec §78, §79, REQ-AGT). Agents act on behalf of a human and never exceed them: effective permissions are a subset of the principal's (AGT-001), every agent has exactly one (<i>unwired</i>)	0	0	166
chatUiBlocks engine.chatUiBlocks	Main / ai	Sanitiser for the envelope's optional "blocks" array (Plexii P4) — the interactive UI blocks the model emits inside an answer. Dependency-free (no Electron, no DB, no SDK) so it unit-tests i	1	1	165
browsing engine.browsing	Main / db		5	4	165
changeLog engine.changeLog	Main / db	WS01 sync substrate — the local end of the change log. The renderer's sync engine records every widget CRDT event here (via the crdt: IPC) so it survives a reload and, crucially, so events e	6	2	165
intentClassify engine.intentClassify	Main / ai	The capture classifier (Attention S5) — hard rules first, Haiku fallback, to_remember floor.	1	1	159
liveDesks engine.liveDesks	Main / db	Local record of which desks are published as public live web views. Signal is the authority on the projection itself; this table is the desktop's side of the arrangement: which desk maps to	1	1	158
localAiSweeps engine.localAiSweeps	Main / ai	Background local-AI passes: document enrichment and memory extraction. Both capabilities were already built, already consumed by retrieval, and free (they run on the local model).	1	1	157
modelRouting engine.modelRouting	Main / ai	Model IDs — kept in main process so renderer never sees raw IDs. These are THE place model versions are named.	16	17	157
localModel engine.localModel	Main / ai	Local model client (Ollama). Runs chat completions and text embeddings against a local Ollama server so document enrichment and semantic indexing happen privately, offline and for free — no	7	5	156
dailyBriefContext engine.dailyBriefContext	Main / ai	Pure assembly of the real environment state into a compact context block for the Daily Brief. Kept side-effect free so it is unit-tested directly.	4	3	148
creationGate engine.creationGate	Main / ai	The discovery creation gate (A4, AI-08 — ruling R8). "No premature desk creation. The conversation grows until Caleb is deliberately prompted ('want me to create a desk?') or uses a persiste	1	1	147
filesFromUrl engine.filesFromUrl	Main / db	Turning a linked image into an owned one. A picture dragged out of a chat or a web page arrives as a URL, and the desk stores the URL.	4	1	147
migrateBrowsingHistoryCounts engine.migrateBrowsingHistoryCounts	Main / db	DEC-061 — repair browsing_history.visit_count. The counts are wrong in the same way, and from the same handler, as the activity-log burst DEC-058 fixed.	1	1	147
widgetLinks engine.widgetLinks	Main / db		9	5	147
signals engine.signals	Main / db		10	3	144
planSelect engine.planSelect	Main / ai	DEC-052 (Track B3, mode b) — the intent-driven planner's SELECTION step. The operator's example: "I'm feeling really motivated to take on the CETRA project today" → compile the relevant open	1	1	143
chatStreamConsumer engine.chatStreamConsumer	Main / ai	The delta → trace-event loop: the piece that turns a stream of text chunks into "the reply landed" and "an action finished being written".	1	1	139
localWhisper engine.localWhisper	Main / ai	Local Whisper — on-device transcription via @xenova/transformers. The engine runs IN THE MAIN PROCESS.	3	3	139
peopleExtract engine.peopleExtract	Main / ai	DEC-088 — capture-time people extraction, deterministic and directory-grounded. "Caleb needs to review this" should arrive at the confirm stop with Caleb already suggested as a person mentio	2	2	139
enrichParse engine.enrichParse	Main / ai	Pure prompt-building and response-parsing for document enrichment. Kept dependency-free (no db, no Electron, no model client) so the exact contract is unit-testable and visible in one place.	1	1	139
meetings engine.meetings	Main / db	—— fb_meetings (PlexiMeet) —— CRUD over recorded / noted meetings. action_items_json is a JSON string array. Mirrors the other local stores's	6	16	137
cacheControl engine.cacheControl	Main / ai	Block-level ephemeral prompt-cache helpers. Prompt caching is GA on the /v1/messages endpoint.	201	2	134
vocabulary engine.vocabulary	Main / ai	The protocol vocabulary quarantine (SPEC-044, Attention layer S0). One word, three meanings, one source of truth.	1	5	134
docMetadata engine.docMetadata	Main / db	—— fb_document_metadata (AI-enriched, local-model generated) —— Structured metadata distilled from a document by the local model.	4	4	124
filesFromDisk engine.filesFromDisk	Main / db	Ingesting files that already exist on a disk. This is the part of the Drive that cannot follow the app into a browser. Not because of effort -- because a tab is never handed a filesystem pat	2	2	123
browserProgress engine.browserProgress	Main / ai	Noticing that a run is going nowhere. Reported as "doing the same thing over and over and costing money to not achieve anything." The loop had no idea. It counted FAILED actions — priorFaile	1	1	123
focusClusters engine.focusClusters	Main / db	SQLite CRUD for Focus-Mode CLUSTERS (split "groups"). Mirrors focusSessions.ts: getDb(), prepared statements, org scoping via getActiveOrgId(), a rowTo* mapper. A cluster is a per-desk (task	3	3	119
agentInputs engine.agentInputs	Main / ai	Turn a wired-in widget into readable content a desk agent can actually reason over.	1	1	118

NAME	MODULE	WHAT IT IS	CALLERS	DEPS	LO
apiTokens engine.apiTokens	Main / db	PlexiAPI token + config store. Tokens are stored only as a sha256 hash; the raw value is generated once, returned once, and never persisted, so a leak of the database cannot reveal a working	6	2	118
browserOverlays engine.browserOverlays	Main / ai	Getting the cookie wall out of the way. Reported as: the run "seems to get stuck with pop ups like cookie consent, doing the same thing over and over and costing money to not achieve anything	2	2	113
migrateNodesKind engine.migrateNodesKind	Main / db	migrateNodesKindCheckV2 — widen the nodes kind CHECK to admit 'work_item' (Attention layer S1; ARCHITECTURE §2.1).	3	4	112
smtp engine.smtp	Main / mail	SMTP send — the outbound half of the user's own mailbox. PlexiDesk reads over IMAP and sends over SMTP using the SAME account: same login, same app-specific password, just the sending server	2	2	112
memoryExtract engine.memoryExtract	Main / ai	Pure prompt-building + parsing for local-model memory extraction. Dependency- free so it unit-tests directly; the model call + store live in extractMemory.ts .	2	2	106
evals engine.eval s	Main / ai	AI evaluation framework (spec §70, §72, REQ-AI/AGT/ENG). Provider substitution is verifiable by an evaluation suite run against every supported model, and a provider is not "supported" witho (<i>unwired</i>)	0	0	103
templates engine.templates	Main / db		5	9	103
embeddings engine.embeddings	Main / db	—— fb_embeddings (semantic-retrieval foundation) —— A vector store keyed by (item_type, item_id).	6	7	100
liveResume engine.liveResume	Main / ai	Live resume-summary generation (spec §52 stage-model, §67, §70). Bridges the deterministic Resume to a real model call through the single model-client seam (PLX-AI-001), staying inside the g	1	1	104
searchText engine.searchText	Main / db	Pure helpers for global search: turn stored content (raw text or a JSON body like a Tiptap doc, a sheet, or a slide deck) into searchable plain text, build a readable snippet around a match,	3	1	103
migrateIntentTaxonomy engine.migrateIntentTaxonomy	Main / db	migrateIntentTaxonomyV2 — rewrite stored intent_class values to the eight aligned primaries (taxonomy alignment stage; DEC-029a sequencing, analysis/22 §3 "the rename migration is a real sta	3	1	103
database engine.database	Main / db	The outcome of the nodes-kind widening on THIS boot, queryable by the sync status surface: a 'no-check-clause' skip means the local DB never became work_item-capable and must be surfaced lou	77	71	97
wireRuns engine.wireRuns	Main / db	Durable history of reactive-wire writes (transform / mirror) into text targets. Each row stores the target's content before and after the write, so the user can see what an automation did an	1	1	97
resumeSummary engine.resumeSummary	Main / ai	The AI stage over a Resume (spec §52, §70) — deterministic first, AI second. The structured Resume is already complete; this stage adds prose over it, and it is safe by construction: - cache (<i>unwired</i>)	0	0	95
focusSessions engine.focusSessions	Main / db		3	2	94
memoryPurge engine.memoryPurge	Main / db	D2 purge machinery (DEC-021; TRACK-LIFECYCLE L2 step 7). When the operator chooses "Delete everything permanently" for a desk, the desk's MEMORY dies with it: fb_memory rows referencing the	1	1	94
sheetParse engine.sheetParse	Main / ai	Parse the rows matrix out of a Sheets AI-fill reply. Dependency-free + pure so it can be unit-tested without the Anthropic SDK.	1	1	93
portalAggregate engine.portalAggregate	Main / ai	Control room: a portal is a live data SOURCE. Its effective content is the aggregation of the desk it points at, PLUS everything wired into it (so one portal can inherit several desks and fe	1	1	91
apps engine.apps	Main / db	—— fb_apps (PlexiBuild) —— CRUD over no-code apps. components_json is the serialised component stack.	4	5	91
agentBrowser engine.agentBrowser	Main / ai	Browser control for desk agents. When an agent is wired to a browser, we give the model a small set of tools to DRIVE that browser — read the current page, open a URL, or run a web search —	1	1	90
docSnapshots engine.docSnapshots	Main / db	Document version history — the canvas_snapshots pattern applied to office files. A snapshot is the document's full body (as persisted JSON) at a moment in time.	2	2	89
imageDownsample engine.imageDownsample	Main / db	Shrinking an image on the way into the Drive. A photo off a phone is 4000px wide and several megabytes. On a desk it is displayed in a widget a few hundred pixels across, and it has to be sy	3	2	88
chatMentions engine.chatMentions	Main / ai	Render the workspace objects the user referenced with "@" into a prompt block (Phase 4.2). The sibling of chatAttachments , and it inherits that module's one rule: the claim is made ONLY abou	2	1	87
agentEnvelope engine.agentEnvelope	Main / ai	Pure validation + honesty enforcement for the agent-step envelope. Kept free of Electron/SDK deps so it unit-tests directly. The stateful model call + action parsing live in anthropic.ts ; th	3	3	86
retrievalIntent engine.retrievalIntent	Main / ai	Turn-level retrieval gating (A4, AI-10). Until A4, every substantive chat turn ran workspace retrieval and web search ceremonially — right for grounded question-answering, wrong for discover	1	1	86
reconcileOrphans engine.reconcileOrphans	Main / db		1	1	86
extractCommitments engine.extractCommitments	Main / ai	M3 (SPEC-003 §3.6) — the commitment extractor, and the contract that keeps it honest (S3-DEC-021 applied to commitments): every commitment must point at the SEGMENT where it was made.	6	1	84
upcasting engine.upcasting	Main / db	Event schema upcasting (spec §64, PLX-EVT-035/044, DOM-012; ADR-0004). Events are immutable and read forever, so an Event written under an old schema is upcast to the current shape AT READ T	1	1	83
whisperCore engine.whisperCore	Main / ai	The pure heart of local transcription — no electron, no process globals — so it runs identically in the main process AND in the utilityProcess worker, and unit-tests import it directly.	1	1	82
forms engine.forms	Main / db	—— fb_forms (PlexiForms) —— A form is metadata pointing at a backing fb_tables table (its fields = the table's columns, its responses = the	3	6	81